DAT321 DIT843
Quality Assurance and Testing

Introduction Lecture

Francisco Gomes de Oliveira Neto

(he/him)

gomesf@chalmers.se
Computer Science and Engineering Department
Chalmers and the University of Gothenburg

Introduction lecture

- **Who** are we?
- What about **Quality Assurance and Testing**?
- **What** are we going to do? And **why**?
- **How** are we doing this?

© Francisco Oliveira Neto - 2020

Who are we? Teaching Team

Francisco Gomes
(francisco.gomes@cse.gu.se)

Sabina Akbarova
(sabina.akbarova@chalmers.se)

Zhuangzhuang Gong
(gusgongzh@student.gu.se)

© Francisco Oliveira Neto - 2020

Who am I?

- Docent / **Associate Professor** (PhD since 2014)
- **Research:** Software Testing, Gen AI for SE, SE Education

- **Pedagogical prizes:**
  - IT Faculty (2019). IT Student Union (2022), **Chalmers Pedagogical Prize (2025)**
  - Chalmers Equality Award (2022), **Chalmers Equality Prize (2025)**

© Francisco Oliveira Neto - 2020

Why should you study software quality?

- Learn how to **plan and measure quality** of a software product or process.
- **Testing is a necessity** in any modern software development process.
  - Knowing advanced concepts in testing **gives you a competitive edge**.
- Bring awareness to important **human-aspects** related to software quality.

© Francisco Oliveira Neto - 2020

Learning Objectives

Knowledge and understanding

- Explain fundamental **concepts in software quality** (e.g., internal / external quality, as well as quality in use) and quality assurance models.
- Describe how **sustainability** can be seen as a quality attribute in software products.
- Describe and explain definitions and activities related to **software testing**, such as faults, failures levels of testing and test automation.
- Explain current research trends within software quality and testing **(e.g., test flakiness, search-based software testing).**
- Explain the distinction between software **verification and software validation**.
- Name and describe different **testing techniques** and approaches, such as mutation testing, automated test prioritisation, test case generation, exploratory testing, etc.
- Explain the concept of **continuous integration** and relate them to software development processes.

Competence and skills

- Define metrics required to monitor and analyse different **quality attributes** of software products.
- **Measure** quality of software products based on the data from development environments and tools.
- Explain and implement test activities in connection to different levels of testing (e.g., system, integration or unit level tests).
- Define and implement **a test plan or a test scope based on quality goals**. Design appropriate and meaningful tests.
- Apply automated testing technologies on realistic examples using relevant tools (e.g., automated **regression testing** on continuous integration pipelines).
- Use automated measurement systems for measuring quality of software products based on the data from the modern development tools.

Judgement and approach

- Assess the importance of software quality in relation to **time and costs in software development projects**.
- Assess the quality of sets of **tests using software artefacts** (e.g., execution logs, test scripts, history of test executions).
- Relate software quality to **societal aspects** of software development.

© Francisco Oliveira Neto - 2020

Course Schedule – See Canvas

- Lectures: **Tue and Fri (09:15 – 12:00).**
- Practice sessions: **Fri 13 – 15.**
- Open Office: **Thur 11 - 12**
- For topics, see Canvas.

Check TimeEdit for room!!

© Francisco Oliveira Neto - 2020

How will we do this?

- Each lecture will have **theoretical and practical** part that can be:
  - Exercises focused on discussion on choosing different approaches.
  - Demonstration of tools and exercises.

- **The lectures require preparation!**
  - We will begin each lecture with a **quiz** to check your preparation.
  - The quiz is **anonymous and not graded**.

© Francisco Oliveira Neto - 2020

Before we begin...

- **From last year course evaluation:**
  - Format of the lectures using quizzes, discussions and slides.
  - Open-door hours to drop-in and talk with the teacher.
  - Usage of Discord for communication between teachers and students.
- **To improve:** Add recap in quizzes, time management in lectures

❤️ Positive feedback from students.

© Francisco Oliveira Neto - 2020

Examination

- **Scale:** Fail, 3 (Pass), 4 (Pass with credit), 5 (Pass with distinction).
  - Final course grade: *0.4 * Assignment Grade + 0.6 * Exam Grade*
- **2 Assignments:**
  - Assignment 1: Quality measures and technical debt. (Individual)
  - Assignment 2: Automated testing techniques and evaluation. (Group)
  - **Each assignment has three attempts**.
  - Re-exam deadlines will be posted in Canvas.
- Examination: **Written exam (on campus).**
- **After Dec. 20th, we will not have lectures (Christmas break)**

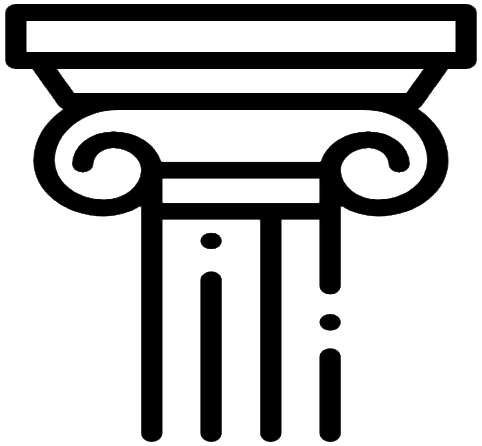
© Francisco Oliveira Neto - 2020

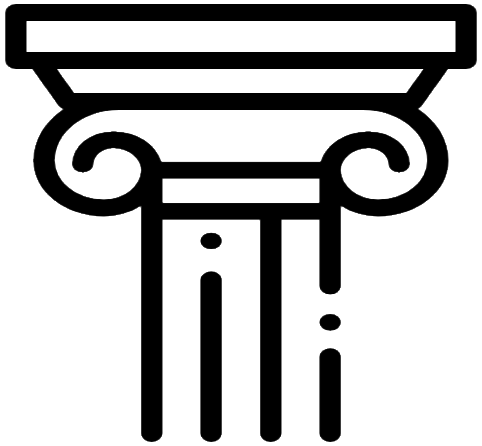
The course has three pillars

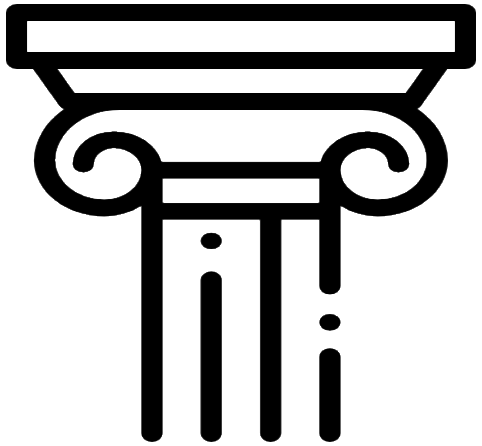
Lecture
Preparation

Lectures

Practice
Sessions







© Francisco Oliveira Neto - 2021

How will the course work?

- All activities are **on campus**.
- No recording allowed of any way.

Communication

Lecture Material and Preparation

Course Information and Announcements

© Francisco Oliveira Neto - 2021

Reading Assignments - Perusall

- Every week you will see **new material available in Perusall**.
- **Collaborative** reading and discussion tool. **Annotate** the material, **ask** or **answer** questions.
- Goals are to **stimulate discussion** by (i) posting questions or comments and (ii) to **help others** by answering their questions.
  - **Effective annotations:** Text contributions that deeply **engage points** in the readings, stimulate discussion, offer informative questions or comments, and help others by addressing their questions or confusions.
- Important: **Perusall is a safe space.** So do not feel hesitant to ask any question.

© Francisco Oliveira Neto - 2021

1.2.4 The myth of "correctness by design"

Now that you have a clearer picture of what I mean by effective and systematic software testing, let me debunk a myth. There is a perception among software developers that if you design code in a simple way, it will not have bugs, as if the secret of bug-free code is simplicity.

Empirical research in software engineering has repeatedly shown that simple, non-smelly code is less prone to defects than complex code (see, for example, the 2006 paper by Shatnawi and Li). However, simplicity is far from enough. It is naive to believe that testing can be fully replaced by simplicity. The same is true for "correctness by design": designing your code well does not mean you avoid all possible bugs.

1.2.5 The cost of testing

You may be thinking that forcing developers to apply rigorous testing may be too costly. Figure 1.4 shows the many techniques developers have to apply if they follow the flow I am proposing. It is true: testing software properly is more work than not doing so. Let me convince you why it is worth it:

- The cost of bugs that happen in production often outweighs the cost of prevention (as shown by Boehm and Papaccio, 1988). Think of a popular web shop and how much it would cost the shop if the payment application goes down for 30 minutes due to a bug that could have been easily prevented via testing.
- Teams that produce many bugs tend to waste time in an eternal loop where developers write bugs, customers (or dedicated QAs) find the bugs, developers fix the bugs, customers find a different set of bugs, and so on.
- Practice is key. Once developers are used to engineering test cases, they can do it much faster.

1.2.6 The meaning of effective and systematic

I have been using two words to describe how I expect a developer to test: *effectively* and *systematically*. Being *effective* means we focus on writing the right tests. Software testing is all about trade-offs. Testers want to maximize the number of bugs they find while

How do we define what code is well-designed if there is always going to be a possibility of bugs? Is there a standard where we then can say that this particular code is designed well? For example, based on how many bugs it lets through or based on the severity of the bugs?

Nov 13 10:30 pm

I'd assume it mainly has to do with a lack of code smells, readability, and such, though I'm not sure if there's any strict way to classify code as well-designed or not.

+2

Nov 14 2:09 pm

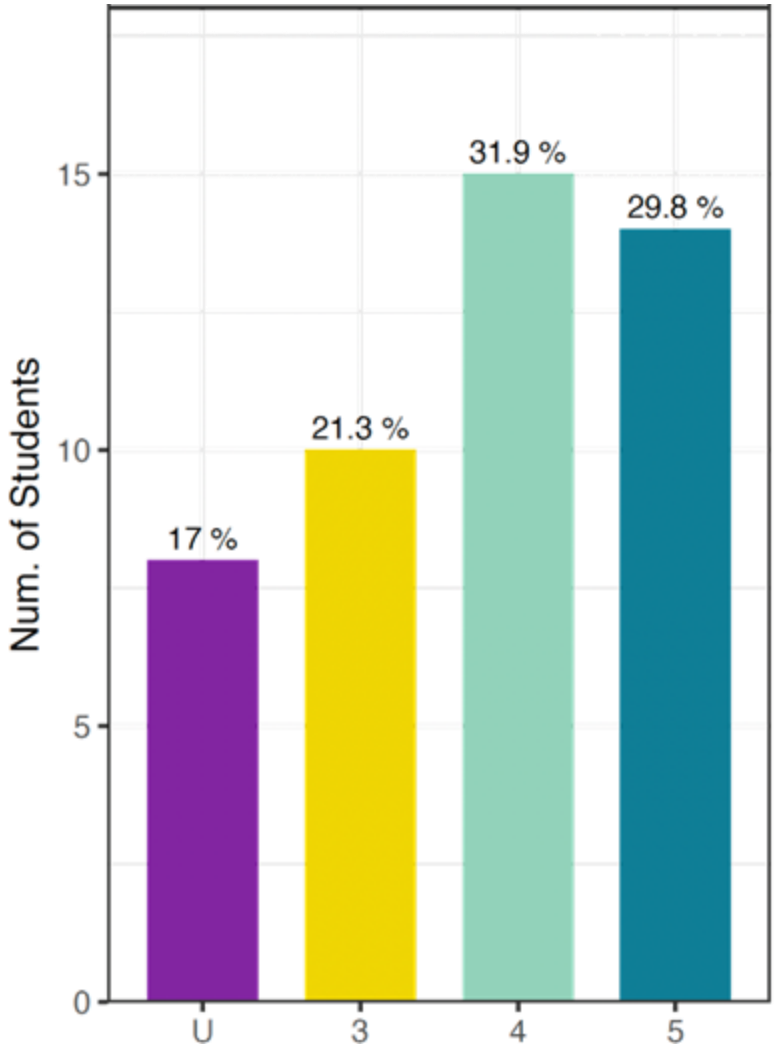
I agree with the last comment. Additionally, I think for example that following design patterns in your code argue for it being "well designed", but what I think the author wants to warn us about is that applying design patterns is not a free ticket to a bug-free code. You can't totally rely on "good practices" (even though they help)- you have to take control of your code and do tests etc.

+1

Nov 14 4:47 pm

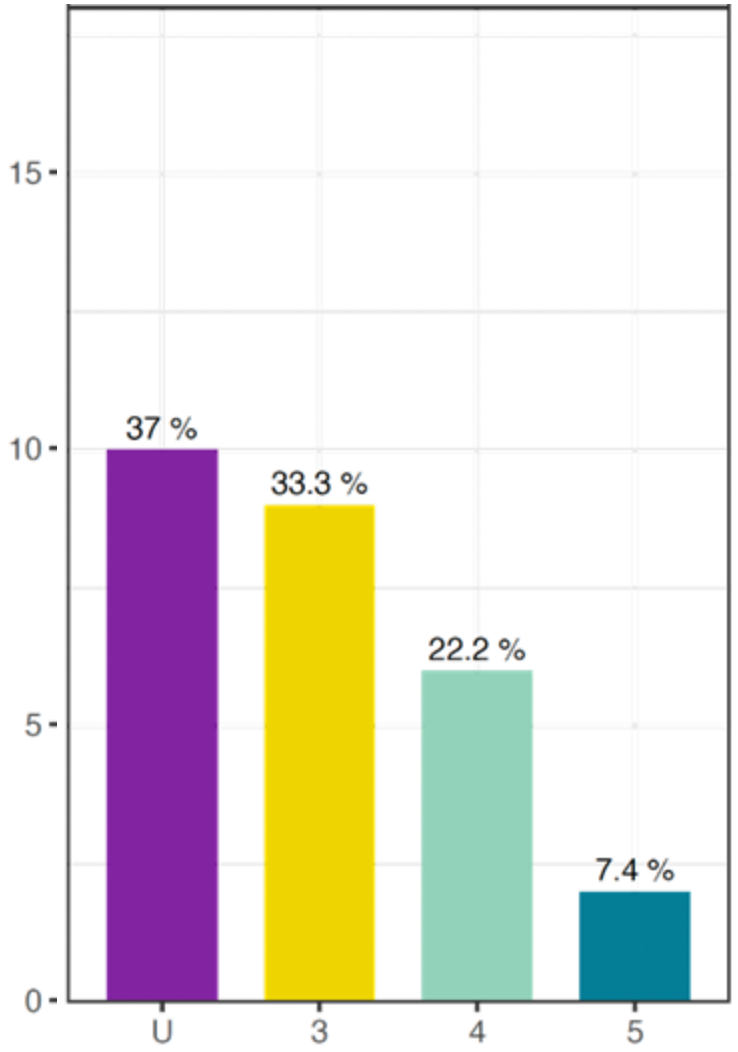
I think of using good open source code for your own project. While the code that you are using might be well designed and good in all ways, it may not fit your needs or your particular configuration or whatever it may be.

Engaged in Perusall (47)

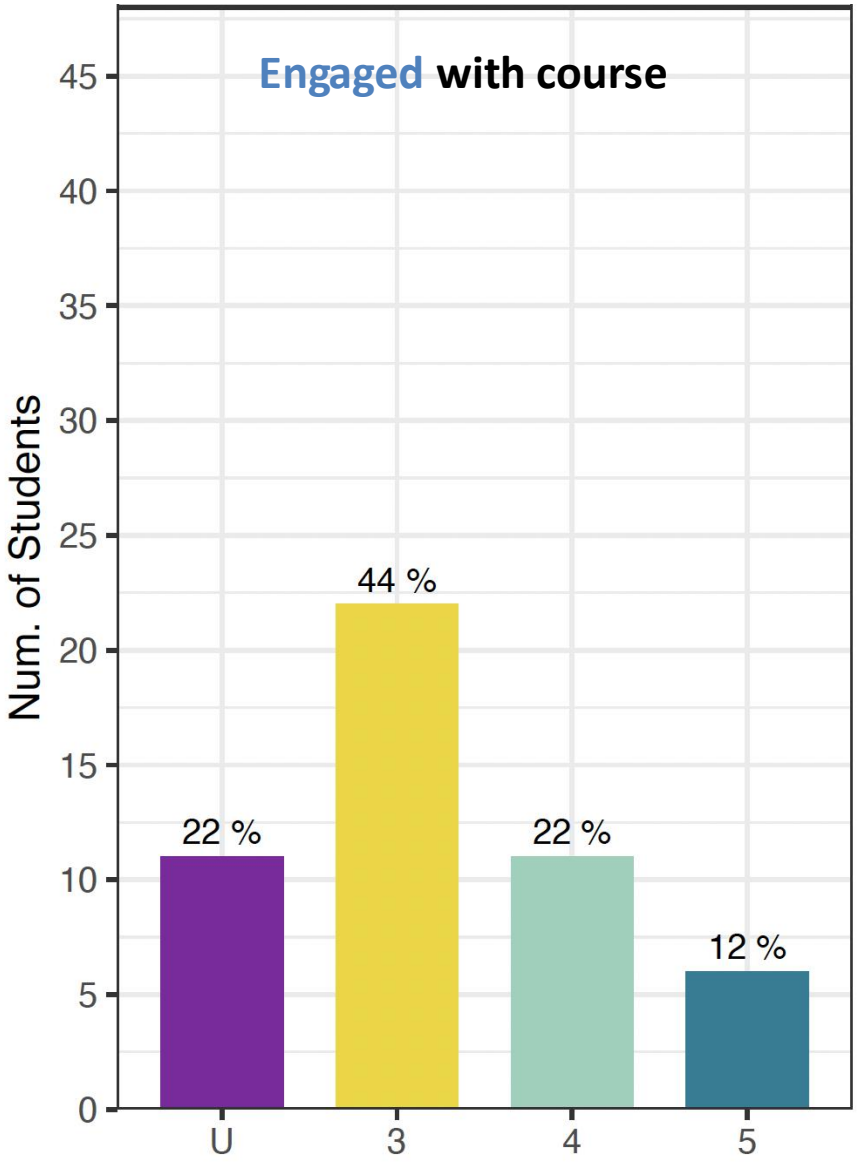


Engagement is key to your Success (MSc Course)

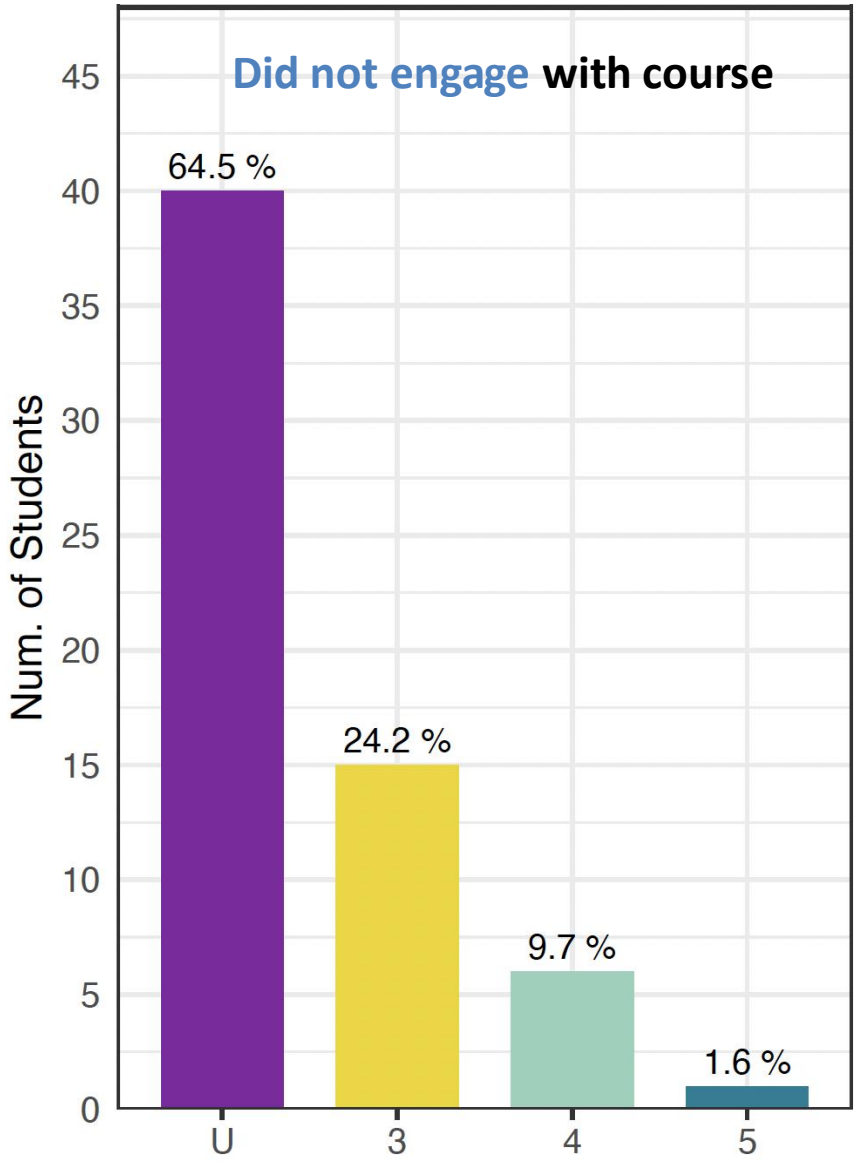
Did not engage in Perusall (27)



© Francisco Oliveira Neto - 2022



Engagement is key to your Success (BSc course)



© Francisco Oliveira Neto - 2022

Reading Assignment - Research on SE Education

- We are researching how to **improve teaching in SE**. Your **annotations are invaluable** for us to understand your learning process. ☺

- We **ask for your consent** to use your annotations in the material for research.
- Completely optional, **sign up via Canvas**. We are thankful for your contribution to science.
- This will **not affect your grade** in any way.
- All **annotations will be anonymised** and your privacy will be protected.

© Francisco Oliveira Neto - 2021

Policies – Code of Conduct

- **Integrity and Ethics:** Homework and programs you submit for this class **must be entirely your own**. If this is not absolutely clear, then contact me. Any other collaboration of any type on any assignment is not permitted. It is your responsibility to protect your work from unauthorized access. If you violate academic integrity in our course, you will fail the course and be reported.

- **Classroom Climate:** Be **respectful** during lectures. Everyone is welcome to join discussion and offer their input. **There are no stupid questions.** Be assertive (not aggressive, not passive and certainly not passive-aggressive) during discussion and class exercises. Disruptive students will be warned and dismissed

© Francisco Oliveira Neto - 2020

Policies – Code of Conduct

- **Diversity:** Students in this class are expected to work with all other students, regardless of gender, race, sexuality, religion, age, etc. **Zero-tolerance policy for discrimination, harassment or micro-aggressions.** Everyone should feel heard and seen.

- **Accessibility:** We will provide reasonable accommodations to students that have disabilities. Contact teaching team early to discuss individual needs. Feedback about accessibility of the material is useful. ❤️

© Francisco Oliveira Neto - 2020

Policies – Generative AI

- We have rules on how ChatGPT, Github Copilot, etc. can be used in the course.

- See Canvas for Details

© Francisco Oliveira Neto - 2020

Student's guidelines

- Check the course webpage: **Canvas and Slack.**
- **Ask in person** during the lectures/breaks. No need to be shy. :)
- Open door at **Francisco's office** every week!
- **Bring questions** and engage in discussions.

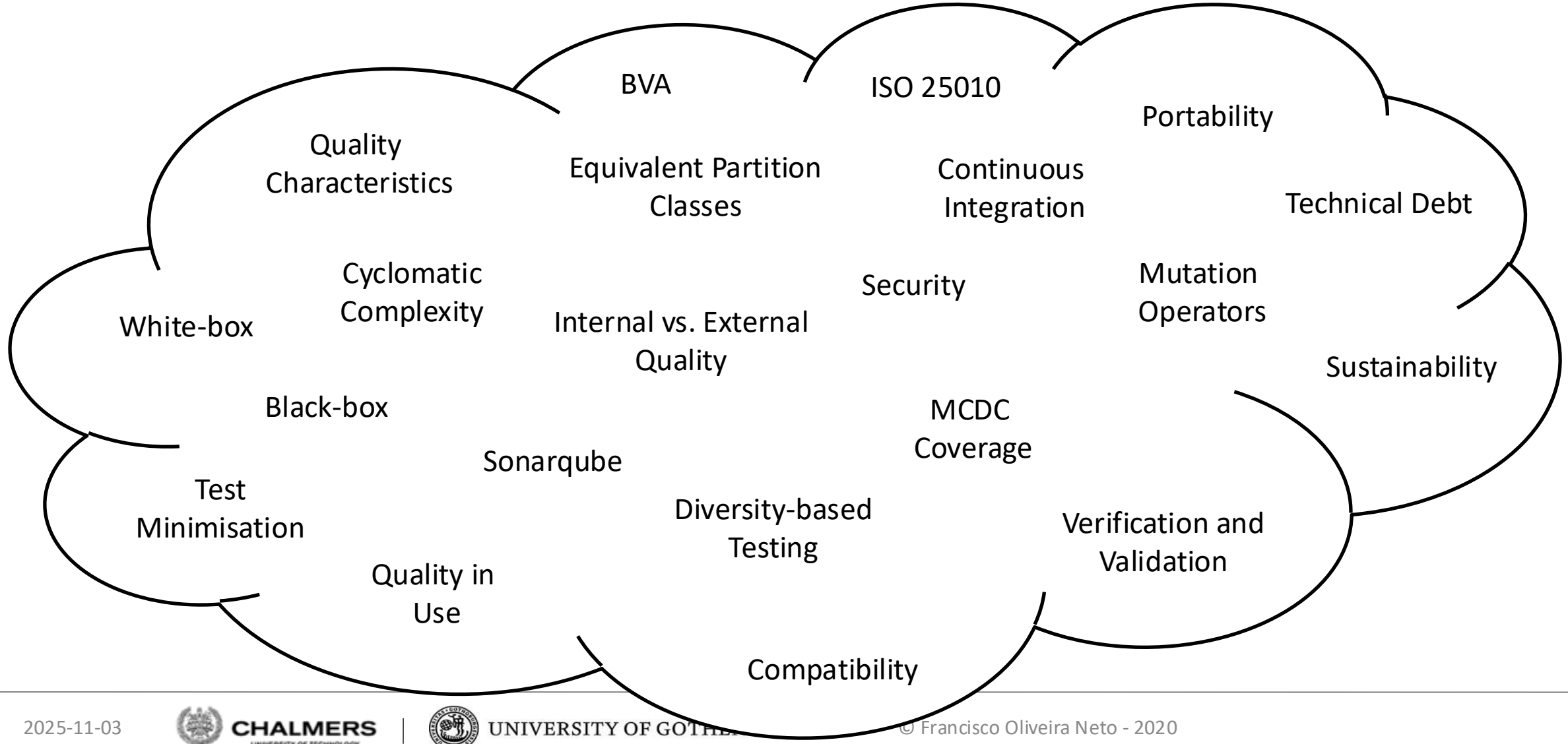
© Francisco Oliveira Neto - 2020

Course Evaluations

- **Student representatives** Group: 2 – 5 students.
  - Send e-mail Francisco including: (i) full name, (ii) GU or Chalmers?, (iii) why you want to be a student representative.
  - Aim for diversity: 50/50 GU + Chalmers students
  - **Deadline: Volunteer by 2025-11-10 (next Monday)**

© Francisco Oliveira Neto - 2020

This season of Quality Assurance and Testing



© Francisco Oliveira Neto - 2020